

Microservices with ASP.NET Core Controller based Web APIs - Resilience

What is Resilience?

Resilience is the process and outcome of successfully adapting to difficult or challenging life experiences, especially through mental, emotional, and behavioral flexibility and adjustment to external and internal demands. In general terms resilience is the general capacity to withstand, recover, or bounce back from difficult events, stress, or change.

Microservices operate in distributed, network-dependent environments where failures are inevitable. Common failure types include network timeouts, dependency outages, resource exhaustion, and configuration issues. Well-isolated services with observability and budget controls benefit most from resilience patterns, while tightly coupled systems see diminishing returns.

Resilience in microservices is a system's ability to handle dependency failures, network disruptions, and heavy loads gracefully without crashing the entire application.

Core Principles

Before diving into specific patterns, understand these foundational concepts:

1. **Transient faults** are short-duration failures (network blips, temporary service unavailability) that often resolve themselves
2. **Cascading failures** occur when one service's failure propagates through dependencies
3. **Fail fast** is often better than slow failure—timeouts prevent resource exhaustion

Understand a Practical Scenario

In a monolithic application, one application may contain most of the functionality:

Angular → ASP.NET Core API → SQL Server

Where as in Microservices, one request may cross several services:

Angular → Ocelot API Gateway → Auth Service
→ Employee Service
→ SQL Server
→ Insurance Service
→ MongoDB

Now imagine: Angular → EmployeeService → InsuranceService is DOWN

Without resilience, EmployeeService may:

- wait indefinitely
- throw exceptions
- consume threads
- retry aggressively
- overload the failing service
- eventually become unavailable itself

This is called **cascading failure**. Resilience patterns attempt to prevent that.

NOTE: Resilience is not one library or one pattern. It is a collection of techniques that allow a distributed system to survive partial failures.

The major resilience patterns

For ASP.NET Core Microservices, few of recommend patterns are listed below:

Pattern	Purpose	Typical .NET technology
Timeout	Don't wait forever	Microsoft.Extensions.Http.Resilience
Retry	Recover from transient failures	Polly / Http.Resilience
Circuit Breaker	Stop repeatedly calling unhealthy service	Polly
Fallback	Return alternative response	Polly
Rate Limiting	Protect a service from excessive requests	ASP.NET Core Rate Limiting
Bulkhead Isolation	Prevent one dependency consuming all resources	Polly / concurrency limiting
Hedging	Try another endpoint when one is slow	Polly
Load Balancing	Distribute calls across instances	Service Discovery / HttpClient
Health Checks	Determine service health	ASP.NET Core Health Checks
Caching	Avoid unnecessary dependency calls	IMemoryCache / distributed cache
Idempotency	Safely handle repeated requests	Application design
Queue-based buffering	Decouple services	RabbitMQ / MassTransit
Backpressure	Prevent overload from propagating	Queues/concurrency limits

Pattern 1: Timeout → "Do not wait longer than X seconds for this dependency."

Key idea behind the pattern is never allow a slow downstream microservice to consume resources indefinitely. After a defined duration, stop waiting and handle the timeout gracefully. In the example walkthrough we will be using **Microsoft.Extensions.Http.Resilience**, which fits well with the resilience approach we've been covering.

Walkthrough:

- Create a blank solution **ResilienceTimeoutDemo**
- Create two ASP.NET Core Controller Based Web APIs
 - InsuranceService → **https://localhost:5102**
 - EmployeeService → **https://localhost:5101**
- **EmployeeService** makes an HTTP request to **InsuranceService**, here only for demo purpose we simulate slow processing to get response after 10 seconds.
- Our Employee API will allow only 3 seconds for the Insurance API. It means after 3 seconds **Timeout** is declared and exception **503 Service Unavailable** is raised.
- Add a controller to InsuranceService: **Controllers/InsuranceController.cs**

```
using Microsoft.AspNetCore.Mvc;
namespace InsuranceService.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class InsuranceController : ControllerBase
    {
        [HttpGet("{id}")]
```

Free, Open-Source, Cross-Platform Framework

```
public async Task<IActionResult> GetInsurance(int id,
CancellationTok cancellationToken)
{
    // Simulate a slow downstream service
    await Task.Delay(10000);

    var insurance = new
    {
        InsuranceId = id,
        PolicyNumber = "POL1001",
        PolicyType = "Health Insurance",
        Premium = 15000
    };

    return Ok(insurance);
}
}
```

- Run InsuranceService on <https://localhost:5102>
- You can test → GET <https://localhost:5102/api/insurance/1>
- Let's work with EmployeeService
- Install Microsoft.Extensions.Http.Resilience
- You can use NuGet Package Manager Console

PM> Install-Package Microsoft.Extensions.Http.Resilience

- This library provides the modern .NET resilience pipeline for HTTP clients.
- Add Services folder to project and create Insurance Client Interface
- Services/IInsuranceClient.cs

```
namespace EmployeeService.Services
{
    public interface IInsuranceClient
    {
        Task<object?> GetInsuranceAsync(int insuranceId);
    }
}
```

- Add respective class: Services/InsuranceClient.cs

```
namespace EmployeeService.Services
{
    public class InsuranceClient : IInsuranceClient
    {
        private readonly HttpClient _httpClient;
        public InsuranceClient(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }
        public async Task<object?> GetInsuranceAsync(int insuranceId)
        {
            var response = await
                _httpClient.GetAsync($"api/insurance/{insuranceId}");
            response.EnsureSuccessStatusCode();
            return await response.Content.ReadFromJsonAsync<object>();
        }
    }
}
```

Free, Open-Source, Cross-Platform Framework

- Notice that the controller does **not** directly call HttpClient, instead:

Controller → IInsuranceClient → InsuranceClient → HttpClient → InsuranceService

- This is a much better microservice architecture.
- Configure HttpClient and Timeout in EmployeeService/Program.cs

```
builder.Services.AddHttpClient<IInsuranceClient,
    InsuranceClient>(client =>
    {
        client.BaseAddress = new Uri("https://localhost:5102/");
    }).AddResilienceHandler("insurance-timeout", pipeline =>
    {
        pipeline.AddTimeout(TimeSpan.FromSeconds(3));
    });
```

- Create Employee Controller Controllers/EmployeeController.cs

```
using EmployeeService.Services;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeService.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeesController : ControllerBase
    {
        private readonly IInsuranceClient _insuranceClient;

        public EmployeesController(IInsuranceClient insuranceClient)
        {
            _insuranceClient = insuranceClient;
        }

        [HttpGet("{id}/insurance")]
        public async Task<ActionResult> GetEmployeeInsurance(int id)
        {
            var insurance = await _insuranceClient.GetInsuranceAsync(id);
            return Ok(new
            {
                EmployeeId = id,
                EmployeeName = "Ravi",
                Insurance = insurance
            });
        }
    }
}
```

- Handle the Timeout
- For a production-quality API, we don't want an unhandled exception. We can explicitly catch the timeout. So, update the client:

```
using EmployeeService.Services;
using Microsoft.AspNetCore.Mvc;
using Polly.Timeout;

namespace EmployeeService.Controllers
```

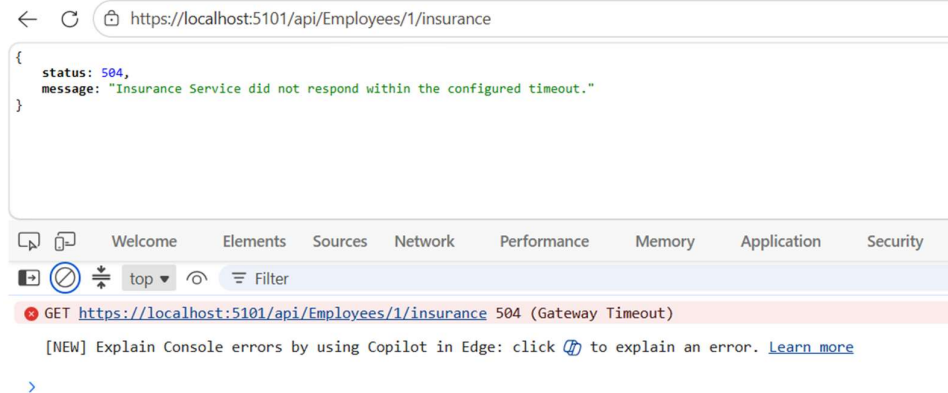
```
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeesController : ControllerBase
    {
        private readonly IInsuranceClient _insuranceClient;

        public EmployeesController(IInsuranceClient insuranceClient)
        {
            _insuranceClient = insuranceClient;
        }

        [HttpGet("{id}/insurance")]
        public async Task<IActionResult> GetEmployeeInsurance(int id)
        {
            try
            {
                var insurance = await
                _insuranceClient.GetInsuranceAsync(id);

                return Ok(new
                {
                    EmployeeId = id,
                    EmployeeName = "Ravi",
                    Insurance = insurance
                });
            }
            catch (TimeoutException)
            {
                return StatusCode(
                    StatusCodes.Status504GatewayTimeout,
                    new
                    {
                        Status = 504,
                        Message = "InsuranceService did not respond within
the allowed time."
                    });
            }
            catch (TimeoutRejectedException)
            {
                return StatusCode(
                    StatusCodes.Status504GatewayTimeout,
                    new
                    {
                        status = 504,
                        message = "Insurance Service did not respond
within the configured timeout."
                    });
            }
        }
    }
}
```

- However, an even cleaner approach is to allow the exception to propagate and handle resilience-related failures at an appropriate API boundary.
- Expected Response after approximately 3 seconds:



- Timeout Does Not Mean the Downstream Service Stops. It is an important point to consider. Suppose:

EmployeeService timeout = 3 sec
InsuranceService processing = 10 sec

- At 3 seconds:

EmployeeService X stops waiting

- But that does not necessarily mean the server-side operation magically disappears. Depending on cancellation propagation and how the downstream operation is implemented, the downstream service may continue processing.
- Therefore, Timeout primarily limits how long the caller waits; cancellation should also be designed properly when you need downstream work to stop.
- Better way to handle Timeout using Polly's TimeoutRejectedException

```
[HttpGet("{id}")]
public async Task<IActionResult> GetInsurance(int id, CancellationToken
cancellationToken)
{
    await Task.Delay(TimeSpan.FromSeconds(10), cancellationToken);

    var insurance = new
    {
        InsuranceId = id,
        PolicyNumber = "POL1001",
        PolicyType = "Health Insurance",
        Premium = 15000
    };
    return Ok(insurance);
}
```

- Timeout means: "I am no longer willing to wait."
- Cancellation means: "Stop the operation if cancellation has been requested."

Imagine your real architecture:

Angular → Ocelot Gateway → EmployeeService → InsuranceService → Database

Free, Open-Source, Cross-Platform Framework

Without timeouts:

```
Angular
|
EmployeeService
|
InsuranceService
|
Database
|
| slow
| slow
| slow
```

With timeout:

```
Angular
|
EmployeeService
|
InsuranceService
|
|---- 3 seconds ----X
|
504 Gateway Timeout
```

Pattern 1: Timeout → "Do not wait longer than X seconds for this dependency."

Pattern 2: Retry → "If the call fails/transiently times out, try again."

Better

Timeout + Retry: "Give each attempt a limited amount of time, and retry the operation when appropriate."

The **Timeout + Retry** prevents waiting forever. Retry attempts recovery.

Suppose:

InsuranceService → Request → EmployeeService → X (Down) temporary failure

A transient failure might be caused by:

- temporary network problem
- connection reset
- service temporarily overloaded
- database connection temporarily unavailable

Instead of immediately failing:

- Attempt 1 -> Failed
- Attempt 2 -> Success

The Retry pattern handles transient faults by automatically re-executing failed operations. Key configuration options include retry count, retry interval, and backoff strategy.

Scenario

EmployeeService → calls InsuranceService → InsuranceService intentionally delays → HttpClient resilience pipeline applies Timeout + Retry → final Timeout is handled by EmployeeController and returned as HTTP 504.

Walkthrough

- Create a blank solution ResilienceTimeoutRetry
- Create ASP.NET Core Controller Based Web API: InsuranceService
- Add to project a Models folder and Insurance model: **Models/Insurance.cs**

```
namespace InsuranceService.Models
{
    public class Insurance
    {
        public int Id { get; set; }
        public int EmployeeId { get; set; }
        public string EmployeeName { get; set; } = string.Empty;
        public string InsuranceType { get; set; } = string.Empty;
        public decimal Premium { get; set; }
    }
}
```

- Add API controller: **Controllers/InsuranceController.cs**

```
using InsuranceService.Models;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace InsuranceService.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class InsuranceController : ControllerBase
    {
        [HttpGet("{id}")]
        public async Task<IActionResult> GetInsurance(int id)
        {
            // Intentionally delay the response
            // to demonstrate timeout + retry.
            await Task.Delay(TimeSpan.FromSeconds(5));

            var insurance = new Insurance
            {
                Id = id,
                EmployeeId = 1,
                EmployeeName = "Ravi",
                InsuranceType = "Health Insurance",
                Premium = 15000
            };

            return Ok(insurance);
        }
    }
}
```

- Configure InsuranceService Port: modify \Properties\launchSettings.json

```
"launchUrl": "swagger",
"applicationUrl": "https://localhost:5201;http://localhost:5217",
"environmentVariables": {
```

- Run/ Verify API: GET <https://localhost:5201/api/Insurance/1>
- Observe JSON Output

Free, Open-Source, Cross-Platform Framework

- Create ASP.NET Core Controller Based Web API: EmployeeService
- Install Microsoft.Extensions.Http.Resilience
- You can use NuGet Package Manager Console

PM> Install-Package Microsoft.Extensions.Http.Resilience

- Create Employee Model: **Models/Employee.cs**

```
namespace EmployeeService.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Department { get; set; } = string.Empty;
    }
}
```

- Create Insurance Service Interface: **Services/IInsuranceService.cs**

```
namespace EmployeeService.Services
{
    public interface IInsuranceService
    {
        Task<string> GetInsuranceAsync(int employeeId);
    }
}
```

- Create Typed HttpClient: **Services/InsuranceServiceClient.cs**

```
namespace EmployeeService.Services
{
    public class InsuranceServiceClient : IInsuranceService
    {
        private readonly HttpClient _httpClient;

        public InsuranceServiceClient(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        public async Task<string> GetInsuranceAsync(int employeeId)
        {
            var response = await _httpClient.GetAsync(
                $"api/Insurance/{employeeId}");
            response.EnsureSuccessStatusCode();

            return await response.Content.ReadAsStringAsync();
        }
    }
}
```

- Configure Timeout + Retry. This is the key step. Open: **EmployeeService/Program.cs**

```
builder.Services.AddHttpClient<IInsuranceService, InsuranceServiceClient>(
    client =>
    {
        client.BaseAddress = new Uri("https://localhost:5201/");
    })
```

Free, Open-Source, Cross-Platform Framework

```
.AddResilienceHandler("insurance-resilience", pipeline =>
{
    pipeline.AddRetry(new HttpRetryStrategyOptions
    {
        MaxRetryAttempts = 2,

        Delay = TimeSpan.FromSeconds(1),

        BackoffType = DelayBackoffType.Constant,

        ShouldHandle = new PredicateBuilder<HttpResponseMessage>()
            .Handle<HttpRequestException>()
            .HandleResult(response =>
                (int)response.StatusCode >= 500)
    });

    pipeline.AddTimeout(TimeSpan.FromSeconds(3));
});
```

- Add API controller to EmployeeService: **Controllers/EmployeesController.cs**

```
using EmployeeService.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Polly.Timeout;

namespace EmployeeService.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeesController : ControllerBase
    {
        private readonly IInsuranceService _insuranceService;

        public EmployeesController(IInsuranceService insuranceService)
        {
            _insuranceService = insuranceService;
        }

        [HttpGet("{id}/insurance")]
        public async Task<IActionResult> GetEmployeeInsurance(int id)
        {
            try
            {
                var insurance =
                    await _insuranceService.GetInsuranceAsync(id);

                return Ok(new
                {
                    employeeId = id,
                    insurance
                });
            }
            catch (TimeoutRejectedException)
            {
                return StatusCode(StatusCodes.Status504GatewayTimeout, new
                {
                    message = "InsuranceService did not respond within the
configured timeout."
                });
            }
        }
    }
}
```

Free, Open-Source, Cross-Platform Framework

```
}  
catch (HttpRequestException)  
{  
    return StatusCode(StatusCodes.Status503ServiceUnavailable,  
        new  
        {  
            message = "InsuranceService is currently unavailable."  
        });  
}  
}  
}
```

☠️😊🌐 Failures Welcome. Downtime Denied. 🌐😊☠️